

JUNE 2026 EDITION

Loop Engineering

Stop Asking Me What It Is

From hand-feeding prompts to designing systems that put
your agents to work

The New Layer Above the Harness

Version: v260615

Released: June 2026

Inside: Definition • Evolution • Anatomy • Six Parts • The Evaluator • Real Loops • The Costs • Getting Started

HuaShu

花叔

Product details, features, and pricing in this book may change at any time. Always refer to each product's official documentation.

Contents

目录

Part 1 • What It Is

§01 Stop Asking: What Loop Engineering Really Is

§02 From Prompt to Context to Loop

Part 2 • How It Turns

§03 The Five Moves of One Loop

§04 Six Parts: What a Loop Is Built From

§05 Generator and Evaluator: Why an AI Can't Grade Its Own Code

Part 3 • Where It Runs, What It Costs

§06 Loops That Run While You Sleep: Three Real Ones

§07 The Costs: Verification Debt, Comprehension Rot, Token Blowout

Part 4 • How You Start

§08 Stay the Engineer, Not Just the One Who Presses Go

§09 Build Your First Loop Today

§01 Stop Asking: What Loop Engineering Really Is

Stop Asking: What Loop Engineering Really Is

Another XX Engineering? What's different this time is that it doesn't teach you how to do the work. It tells you to stop doing the work yourself.

Here we go again

The Prompt Engineering courses are still selling, the ink on Context Engineering isn't dry yet, and I just finished a whole book on Harness Engineering. Now Loop Engineering shows up too.

Over the past year a string of "XX Engineering" terms has popped up, almost keeping pace with model releases. You want to roll your eyes. I get it.

But this one is a little different. It's not about doing the work better. **It's about pulling you out of the position of doing the work entirely.**

The earlier terms all assumed you were sitting at the keyboard, directing the agent line by line. Loop engineering deletes that assumption. You're no longer inside the loop. You're outside it, building the loop.

A one-line definition

The person who named the term and wrote it up is Addy Osmani, an engineer on the Google Chrome team. His definition is short, so I'll just give you his words.

Original: Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead.

In plain terms: loop engineering means swapping out "the person who prompts the agent" — from you, to a system. **You no longer feed it line by line; you design the system that feeds it for you.**

The weight of that sentence is on "replacing yourself." It's not about writing a better prompt, or managing context more precisely. It's about moving you, the person, out of that position.

This is a shift of position. You used to be the engine; now you're the person who designs the engine. What you write is no longer the words for the agent — it's a thing that automatically sends words to the agent.

Within one week, three people lit the fuse

This term wasn't dreamed up out of nowhere one day. In one week in June 2026, several groups of people ran into the same thing at almost the same time, and only then did it get a name.

What really set it off was a post from **Peter Steinberger** (the author of OpenClaw) in June. The post shot up past 8 million views.

Original: You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents.

You shouldn't be prompting coding agents anymore. You should be designing the loops that prompt your agents.

At almost the same time, **Boris Cherny** (from Anthropic, the lead on Claude Code) was saying the same thing.

Original: I don't prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops.

I don't prompt Claude anymore. I have a bunch of loops running, and they prompt Claude and figure out what to do. My job is to write loops.

The one who actually named it was **Addy Osmani**, the Google Chrome engineer. On June 7 he wrote it up on his personal blog under the title Loop Engineering, pulling in what Steinberger and Cherny had said, and synced it to Substack the next day. One ignition, one echo, one name — all within a week.

These three lines are saying the same thing

Stack the three of them together and you'll see they're pointing at the same move.

Cherny says "my job is to write loops." Steinberger says "design the loops." Osmani says "design the system that feeds it for you." **The keyword is the same in all three: what you design has shifted from a single behavior of the agent to the entire system that drives the agent.**

That's why three people who weren't comparing notes all landed on the same word in one week. It's not a coincidence. They were each doing similar work, and at some point they realized they needed a shared name to talk about it.

Same as vibe coding back then. You can laugh at how crude the term is, but it did turn a way of working into something you could actually discuss. Loop engineering is the same deal.

From "you prompt the agent" to "you design the system that prompts the agent"

Let me break this shift into the two simplest sentences so it sticks.

The old world: you sit there and prompt the agent line by line. It finishes one thing, stops, and waits for your next line. You're the human clock inside the loop, and every tick has to come from you.

The new world: you design something that ticks on its own, tick after tick. **It runs on a timer, spawns little helpers to do the work, and feeds its own results back to itself.** You're no longer in the loop. You're outside it, watching it turn.

Addy's next line nails the relationship.

Original: Loop engineering sits one floor above the harness.

Loop engineering sits one floor above the harness. The harness below handles arming a single agent run; the loop above makes it run itself over and over. How to build that floor is the subject of the next chapter.

By now you can probably feel that this isn't just another reskinned buzzword. **It marks a change of identity: from the person who operates the agent to the person who schedules the agent.**

Your value used to be in "knowing how to direct." Now the directing is taken over by a loop you wrote yourself. Your value moves to "knowing how to build loops, and how to put a check inside the loop that can say no." Why that last part is the hardest thing of all, we'll get to in time.

§02 From Prompt to Context to Loop

The Triple Jump: From Prompt to Context to Loop

Four layers in the stack, each minding one thing. See the boundaries and you'll know where the loop sits, and why it's the floor on top.

The four-layer stack, laid out first

That string of "XX Engineering" terms the AI world has coined over the past two years aren't replacements for each other. They stack. One on top of the next, each minding something bigger.

Let me put the table up first, then walk through it layer by layer.

Layer	What it minds	Core question
Prompt engineering	Writing one good prompt	What should I tell the model
Context engineering	What goes in the window right now	What to retrieve, summarize, clear out
Harness engineering	Arming a single run	Which tools, which actions allowed, what counts as done
Loop engineering	Scheduling on top of the harness	How to make it run itself over and over

Each layer up, what you have to worry about gets one size bigger. From "one sentence" to "one window," then to "one run," and finally to "a whole loop that runs itself." Let's take them one at a time.

Layer one: Prompt, the words you write for the model

The bottom layer, and the one everyone knows best. Prompt engineering minds "what should I tell the model."

You weigh the wording, give examples, set a role, tune the tone. **Its boundary is clear: it's this one time, the chunk of text you hand the model.** Hand it in, the model spits something out, done.

The trouble with this layer is that it assumes you're present every time, handing it in yourself every time. One prompt solves one thing, and the next thing needs you to write another.

Layer two: Context, what goes in the window right now

One layer up, the question changes. It's no longer "what do I say," but "what should go into this window right now so the model can actually crack the problem."

That's context engineering. What references to retrieve, how to summarize the long stuff, which stale information to clear out. **It minds not your words, but the entire field of view the model can see at the moment.**

This is its own layer because the model's window is limited, and stuffing in the wrong thing is worse than stuffing in too little. Give it a window full of noise and the best prompt in the world is wasted.

Layer three: Harness, the full kit for a single run

Up again, to harness engineering. This layer minds what an agent needs to carry when it runs once.

Which tools to give it, which actions to let it touch, when to load context, how to recover when it fails, what state counts as "done." **The harness only minds what an agent can do in a single run, and where it gets stopped.**

I unpacked this layer across an entire book elsewhere, so I won't repeat it here. Just remember its position: the harness is the layer below the loop. It arms a single run of the agent, but it doesn't make that run repeat itself.

Layer four: Loop, making it run itself over and over

At the top, loop engineering. With the first three layers done, you've got an agent that's fully equipped and can run cleanly once. But it stops when it's done, waiting for you to strike the next tick.

The loop layer is about automating that "waiting for you" away too. Addy's line says it cleanest.

Original: Loop engineering sits one floor above the harness.

Loop engineering sits one floor above the harness. **The floor below arms a single run; this floor makes it run itself over and over.**

As for how it runs, Addy gives a picture too: it runs on a timer, spawns helpers, feeds itself. Three verbs in his words, and I'll explain them one by one.

One floor up, what's actually added

Harness and loop often get talked about as one thing, but what they mind is a floor apart. I'll draw the line using Addy's three verbs.

"Runs on a timer." The harness arms a single run, which you have to trigger. The loop fits it with a timer, so it wakes up on schedule and gets to work, no button-press from you. This is the first sense of "runs itself over and over."

"Spawns helpers." Once a loop is turning, it splits off sub-agents to do parallel work. One drafts a change, another does nothing but pick it apart in review. The harness cares about how a single agent is armed; the loop cares about how a crowd of agents coordinates and who supervises whom.

"Feeds itself." This is the crucial one. What the loop produces becomes its own input for the next round. Yesterday's findings get written into a file; this morning it wakes up, reads that file first, and carries on. **It has memory that spans single conversations, which is why it's called a loop and not a one-off task you happened to run many times.**

Addy has a line that gets at this relationship: automation is what makes a loop an actual loop, not some run you did once by hand. Without "waking itself up and picking up the baton," what's left is just pressing the harness many times by hand.

Why bother splitting it this finely

You might ask, is it really necessary to split it this fine? Isn't it just more and more automated?

It's necessary. **Because each layer fails in a different way, and the check that can say no has to be installed in a different place.**

A prompt goes wrong, you see it on the spot and fix a line. Context loaded wrong, the model answers off, and you'll likely catch that too. But by the loop layer, it runs while you sleep, changes code you never looked at, and feeds its own errors into the next round — and you might not find out for days.

The higher the layer, the farther you are from the scene, and the longer the mistakes pile up. That's exactly why the real difficulty of loop engineering is never building the loop, but putting something inside it that can stop it. This topic is the main event of the chapters ahead.

Remember this stack and you've got enough: prompt minds one sentence, context minds one window, harness minds one run, loop makes it keep running. **Your position is climbing this stack layer by layer, from the person who writes the words to the person who builds the loop.**

§03 The Five Moves of One Loop

The Five Moves of One Loop

Take one turn of a loop apart and there are really only five moves inside. Understand these five and you understand every loop.

The word "loop" is easy to misread. Hear "loop" and what comes to mind is a stretch of code running over and over, spinning without end.

But the loop in loop engineering isn't that idle spin. Every turn, it does something concrete: it finds something worth doing, hands it to the agent, verifies whether the result is right, saves the state, then decides the next step.

Five moves: discovery, handoff, verification, persistence, scheduling. Drop any one and the loop won't turn, or it'll turn but spin in place.

Threading it through one real example

Addy Osmani is on the Google Chrome team, and he built himself a morning triage loop. This example gets cited a lot because it's concrete enough that every move lands somewhere real.

Here's the whole flow. In the morning an automation kicks off on its own, no click from him. A triage skill reads yesterday's failing CI tests, the issues still open, and the recent commits. It writes the triage results into a markdown file or a Linear board. For every finding worth acting on, it opens an isolated worktree. One sub-agent drafts the fix; a second sub-agent reviews it against the project's skills and tests. A connector automatically opens the PR and updates the ticket. Anything it can't handle goes to an inbox to wait for a human. And that state file stays around, so the next day picks up right here.

The beauty of this loop is that no step needs a human's hand, yet it stops to wait for a human exactly where it should. Everything that can be automated is fully automated; everything it can't handle goes honestly into the inbox. Now let's take that one turn apart by the five moves.

Move one: discovery

The first thing a loop does on a turn is figure out "what should this turn do."

In Addy's example, the discovery step is the triage skill's job. It reads three things: CI failures, open issues, recent commits. Together those are "what came up in the system worth minding, between yesterday and today."

The key to discovery is letting the agent find its own work, rather than you feeding the work to it. If you manually list "fix these three bugs today" every morning, the loop is still started by you, and it hasn't saved you anything.

There's a detail here that's easy to miss. Addy stresses that the automation triggers a skill, not a big block of instructions. In his words, you trigger a ``$skill-name``, not a wall of instructions pasted into a cron job nobody will ever update. A skill is knowledge made permanent; when the discovery logic changes, you just change the skill.

How well discovery is done sets the ceiling on the whole loop's quality. **If the work it surfaces has no value, the other four moves can be done beautifully and it's still doing useless work, carefully.** So this step isn't just "read some stuff." It's teaching the agent to judge what's worth doing and what can be left alone — and that's precisely the hardest part to bake into a skill.

Move two: handoff

Once you've found the work, you have to hand it off for someone to do. This step is the handoff: moving the task from the "scheduling system" into the hands of the "agent that does the work."

Handoff isn't just shouting "go fix this." Addy's approach is that every finding worth doing gets its own isolated worktree. A worktree is a git mechanism that lets multiple agents change code in separate working directories without stepping on each other.

Why isolate? The next chapter goes into it. For now, one line to remember: **two agents changing the same file at once is exactly the same headache as two engineers committing to the same lines.** Separate them at handoff time and you've got a lot less conflict to clean up later.

After the handoff, the one doing the actual work is the sub-agent. One drafts the fix and writes a first version of the change. Whether that version is any good isn't for it to say.

There's a design tradeoff worth noticing here. **Handoff isn't dumping the whole job on a single agent; it's cutting it into isolated little tasks first, then handing them out separately.** The cleaner the tasks are cut, the easier verification and merging are later. One finding maps to one worktree, maps to one clear change — clear boundaries, and easy to pin down whoever goes wrong.

Move three: verification

This is the step in the whole loop that's easiest to cut corners on, and the one you can least afford to skip.

Addy's design is that after the first sub-agent drafts the fix, a second sub-agent reviews it. The second agent checks against the project's skills and tests, its instructions differ from the first, and sometimes even the model is swapped out.

Why insist on a different agent for review? Because **the agent that wrote the code is too soft when grading its own homework**. Let it judge what it just wrote and it tends to confidently praise the whole thing — even when, to a human eye, the quality is clearly mediocre. Swap in an agent with different instructions, dedicated to picking holes, and it catches the problems the first agent talked itself into letting through.

Addy puts it bluntly: the hard part of a loop isn't the loop itself, it's putting something inside it that can say no. A loop with no real check is just an agent nodding at itself over and over.

This verification step is that thing that can say no. It decides whether the loop is really doing work or just hypnotizing itself.

Move four: persistence

Once verification passes, the result has to land somewhere that survives this conversation. That's persistence.

Addy uses two things. The change itself, via a connector, automatically opens a PR and updates the ticket, landing in the code repo and the issue system. Findings it can't handle go into the triage inbox, waiting on a human. And there's a state file running through it all, recording "where it's up to."

The point of persistence is that a loop's memory can't live only in the current context window. Clear the context and the agent forgets everything. But what's written into markdown or a board doesn't forget. The next chapter expands on this when it covers components, and there's a good line there: the agent forgets, the repo doesn't.

Without persistence, every turn of the loop is like opening its eyes for the first time, always starting from zero. With it, what didn't get finished today can be picked up tomorrow.

Move five: deciding the next step / scheduling

The first four moves — discovery, handoff, verification, persistence — add up to "one turn." But one turn isn't a loop.

What actually makes it a "loop" is the fifth move: scheduling.

Addy's triage loop runs automatically in the morning, no click from him each day. That state file lets findings not finished today carry over to tomorrow, and tomorrow's turn picks up automatically from right here. **It's scheduling that turns "I ran it once by hand" into "it keeps turning on its own."**

Addy has a line that gets right to it: automation is what makes a loop an actual loop, not just one run you did once.

Original: Automations are what make a loop an actual loop and not just one run you did once. Automation is what makes a loop a real loop, not just the one time you ran it.

Without scheduling, the first four moves can be done beautifully and it's still a one-off manual job. You run it today on a whim and forget it tomorrow, and it stops. Add scheduling and it turns endlessly on a timer, carrying the thing on for you.

The five moves, one line each to remember

Put the five moves together and the loop's skeleton is clear.

Move	What it does	In the triage loop
Discovery	Find this turn's work on its own	skill reads CI failures / issues / commits
Handoff	Hand the task to an agent, isolated	each finding opens a worktree
Verification	Swap in another agent to say no	a second sub-agent reviews against tests
Persistence	Write state outside the conversation	open PR + inbox + state file
Scheduling	Make it turn automatically, round after round	morning automation runs on its own

This chapter took apart the moves — "what happens in one turn." The next chapter takes apart the parts — "what things you need in hand to build a loop that can turn." The two are two sides of one thing: moves describe the running, parts describe the materials.

§04 Six Parts: What a Loop Is Built From

Six Parts: What It Takes to Build a Loop

The last chapter took apart the moves; this one takes apart the parts. Addy groups what it takes to build a loop into six things. Miss any one and the loop limps somewhere.

Moves describe "what happens in one turn"; parts describe "what you need to have in hand for it to turn at all."

The two line up. Discovery runs on skills, handoff on worktrees, verification on sub-agents, persistence on memory, scheduling on automations. This chapter lays out the six parts one by one, each making three things clear: what it is, why you need it, and what Addy says about it.

Part one: Automations (automation / scheduling)

An automation is the thing that gets the loop moving on its own. It hangs off a schedule or a trigger, and when the time comes it discovers and triages automatically, no human standing by to press a button.

Why does it come first? Because it's the root of the word "loop." **Without scheduling, the set of things in your hand is just one run, not a loop.** Addy's triage running automatically in the morning in the last chapter (as described in § 03) is exactly automation at work.

Original: Automations are what make a loop an actual loop and not just one run you did once. Automation is what makes a loop a real loop, not just the one time you ran it.

Note one detail. What the automation triggers should be a skill, not a whole wall of instructions. Paste a big block of prompt into a cron job and nobody will ever update it again; trigger a named skill, and when the logic changes you just change the skill.

Scheduling comes in more than one form. Some runs on your own machine, which has to stay on; some runs in the cloud, turning even when your machine is off. The two differ quite a bit, and a later chapter is dedicated to how to choose your scheduling. For now, just remember: automation is the loop's engine, and if the engine doesn't fire, the other five parts are just decoration.

Part two: Worktrees (git worktree isolation)

A worktree is a built-in git mechanism that lets you open multiple mutually independent working directories in the same repo. Each agent changes code in its own worktree without interfering with the

others.

Why do you need it? Because once a loop runs multiple agents at once, file write conflicts are only a matter of time. Two agents fighting to change the same file — that's a headache you've probably had a taste of.

Original: Two agents writing the same file is the exact same headache as two engineers committing to the same lines.

Two agents writing the same file at once is the exact same headache as two engineers committing to the same lines.

Worktrees separate parallel agents and avoid the conflict before it happens, instead of merging a pile of clashing changes after the fact. In the last chapter's handoff step, each finding opening its own worktree is this part doing its job.

The value of this part scales up with parallelism. A loop running only one agent at a time can take or leave worktrees. But once you want it to fix five bugs in parallel in the morning, with five agents each making their own changes, no isolation means five sets of changes tangled together, and in the end nobody knows which line is whose. Worktrees turn parallelism from "runs but messy" into "runs and clean."

Part three: Skills (reusing knowledge via SKILL.md)

A skill makes project knowledge permanent in a single file (SKILL.md), so the agent can reach for it every time instead of re-deriving the context on every turn.

Why do you need it? Because without making it permanent, every run you have to tell the agent all over again "what this project is, what the rules are, where the traps are." This cost of explaining over and over has a name from Addy: intent debt.

A skill is paying off that debt. Write down what you've thought through once, and every turn afterward skips re-deriving it. Why the discovery step is a triage skill rather than a bare prompt comes down to this too — a skill can be reused and maintained, a wall of prompt can't.

Part four: Plugins & Connectors (MCP connectors)

A connector is the interface that hooks the loop to the outside world. The issue tracker, the database, the staging environment's API, Slack — they all come in through it. The technical foundation is MCP (Model Context Protocol).

Why do you need it? Because a loop that can only see the filesystem can do too little. It has to be able to read tickets, query databases, open PRs, and send notifications before it's really running work for you.

Original: A loop that can only see the filesystem is a tiny loop.

A loop that can only see the filesystem is a very small loop.

MCP has another upside: cross-tool compatibility. **A connector you wrote on the Codex side can often be dropped straight onto the Claude Code side and used as is.** In the persistence step, the connector automatically opening PRs and updating tickets is this part doing the cleanup.

Looked at another way, connectors decide the loop's "radius of vision." Hook up only the filesystem and all it sees is a few directories. Hook up the issue tracker and it knows what work is still undone; hook up the database and it can query real data; hook up Slack and it can tell you the results. Every external system you add stretches the chain the loop can complete on its own.

Part five: Sub-agents (separating the generator and the judge)

The key use of sub-agents is splitting "the one that writes" and "the one that judges" into two. One agent generates; another, with different instructions and sometimes a different model, picks holes.

Why insist on splitting them? Because when one agent is both player and referee, the referee plays favorites.

Original: The model that wrote the code is way too nice grading its own homework. A second agent with different instructions and sometimes a different model catches the stuff the first one talked itself into.

The model that wrote the code is too polite grading its own homework. Swap in an agent with different instructions and sometimes a different model, and it catches what the first one talked itself into.

Verification is trustworthy precisely because the judge and the generator aren't the same one. That "thing that can say no" from the last chapter, mapped to a part, is this second sub-agent.

There's a counterintuitive point worth noting. You might think that to make an agent critique its own work, you have to pour effort into training the generator. But in practice it's the opposite: tuning an independent judge to be picky enough is far easier than getting the generator to be hard on its own work. That's also why a loop would rather keep an extra agent around than let one agent review and audit itself.

Part six: Memory (persistent state)

The sixth part, Addy calls it Memory, not Persistent State. It's the loop's memory that lives outside a single conversation — a markdown file, or a Linear board.

Why do you need it? Because the moment an agent's context window is cleared, it remembers nothing. For a loop to pick up today where it left off yesterday, the memory can't sit only in the context; it has to land on disk.

Original: The agent forgets, the repo doesn't. The memory has to be on disk and not in the context.
The agent forgets, the repo doesn't. The memory has to be on disk, not in the context.

The state file written into markdown or a board in the persistence step is this part. It gives the loop continuity. Without it, every turn opens its eyes anew after amnesia.

There's a subtle but important distinction here. Memory isn't context. Context is what the agent can see this round, and it gets flushed when the window refreshes. Memory is what's written to disk, still there across rounds, across days, across the whole conversation's lifetime. A loop that wants to run long-term doesn't rely on stretching the context bigger; it relies on settling what should be remembered onto disk, then reading it back when needed.

Six-part quick reference

Six parts, mapped to five moves, all in one table.

Part	What it is	Maps to move	One-line quote
Automations	Runs automatically off a schedule/trigger	Scheduling	make a loop an actual loop
Worktrees	Isolated working dirs for parallel agents	Handoff	same headache as two engineers
Skills	Make project knowledge permanent, pay off intent debt	Discovery	fire \$skill-name, not a wall of instructions
Connectors	MCP hookup to external systems	Persistence / Discovery	only see the filesystem is a tiny loop
Sub-agents	Generator separated from judge	Verification	too nice grading its own homework
Memory	Persistent state on disk	Persistence	the agent forgets, the repo doesn't

With all six parts in place, a loop that can turn itself has a skeleton. Automation makes it move, worktrees keep it from fighting itself, skills keep it from redoing work, connectors let it see outside, sub-agents let it correct itself, memory lets it remember.

But building it is only the start. Whether a loop that can turn turns well or badly is another matter. The same set of parts, built by two people, can come out completely opposite. That's the cost and the anti-patterns we'll get into next.

§05 Generator and Evaluator: Why an AI Can't Grade Its Own Code

Generator and Evaluator

The hardest part of a loop isn't getting the agent to run. It's putting something inside that can say "no." And the agent writing the code is exactly the one least likely to say it.

It always praises itself

You have an agent finish a chunk of code, then ask it to grade its own work. What does it say?

Anthropic engineer Prithvi Rajasekaran answered this in an official blog post. Building long-running applications, he kept seeing the same thing.

Original: when asked to evaluate work they've produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre.

Ask an agent to evaluate what it produced and it tends to praise it confidently, even when a human can plainly see the quality is mediocre.

This isn't a smarts problem. It's an agent grading its own homework. The context where the code got written is already stuffed with all the reasons it was written that way. When the agent looks at its own code, it doesn't see the result—it sees the chain of self-persuasion that led there.

You and I do this too. You write something, read it back, and it reads smoother every pass. The problem isn't that the writing is good. It's that you already have a version of what it's supposed to be in your head. When you read, you're not seeing the words on the page—you're seeing that version in your head.

Inside a loop, this flaw gets amplified. A loop is valuable because it can run unattended for many rounds. But if every "is this version good enough" gets decided by the agent that just wrote it, then every round it's nodding at itself. The longer it runs, the more self-praise it piles up, and the further it drifts from real quality. A loop without a real check is, at bottom, an agent repeatedly reaching consensus with itself.

Tuning a skeptic is easier than fixing a modest author

Rajasekaran tried making the generator agent more critical of itself. It didn't work well.

Original: tuning a standalone evaluator to be skeptical turns out to be far more tractable than making a generator critical of its own work.

Tuning a separate evaluator to be skeptical is far easier than getting a generator to critique its own work.

The difference is structural, not a matter of wording. You can't get an author to step outside its own perspective with a "please be stricter." But you can swap in another agent, hand it a completely different set of instructions, and have it look at the code from scratch. It didn't write any of it, so it carries none of the self-persuasion.

This is a counterintuitive result. Our instinct is to fix the prompt: write it meaner, push the generator to find its own faults. But Rajasekaran's experience is that this road gets harder the further you go, and it's far less cost-effective than just spinning up another agent. **Self-critique is hard because the thing being critiqued and the critic share one context. Swapping agents is easy because you've used structure to manufacture an outsider.**

Rajasekaran is blunt about where the idea comes from. He borrowed it from GANs.

Original: I designed a multi-agent structure with a generator and evaluator agent.

In a generative adversarial network (GAN), one network builds and one network picks faults. The two work against each other, and the builder gets better and better at looking real. Ported to agents, it's a generator that writes and an evaluator that reviews—structurally separating "writing" from "judging whether the writing is good."

This is exactly what the sub-agent cell in Addy's framework from the last chapter was about. Addy puts it less gently (his words are in § 04, part five): the model writing code is too easygoing when grading its own homework. Swap in a second agent, a different set of instructions, sometimes even a different model, and it can catch the holes the first agent talked itself into.

The evaluator should act, not just read

Swapping agents isn't enough on its own. If the evaluator only reads the code, it's still judging "does this code look right," not "does it run right."

Rajasekaran's approach on frontend tasks is worth copying. **He hooked the evaluator up to Playwright MCP so it could open the page itself, click buttons, take screenshots, inspect the DOM—use the thing like a real QA person would.**

That single move shifts the basis for judgment from "this JSX looks fine to me" to "I clicked the login button, the page navigated, here's the screenshot." An evaluator that acts looks at behavior, not intent.

One nice detail: Addy mentions that when he swaps agents, he sometimes swaps the model too (sometimes a different model). The same model, even with new instructions, often keeps its blind spots right where they were. Swap the base and even the holes it habitually steps around may change. For judgment, the bigger the difference, the more it's worth.

When writing evaluator instructions, practitioners often calibrate toward the worst case. A common phrasing tells the evaluator to assume "this code is broken until proven otherwise." That line isn't an official Anthropic statement—it's something the community figured out by feel. But the direction is right: **the evaluator's default stance should be doubt, not trust.** The generator already trusts itself plenty. If the evaluator is polite too, the loop is just an agent nodding at itself.

In a product: /goal puts the evaluator on the stop condition

By now you might feel that generator and evaluator sound like two boxes on an architecture diagram, a little far from your hands. In fact Claude Code already has a command that turns this structure into a product primitive: **/goal**.

You use **/goal** by giving an agent a condition and letting it run until the condition is met. For example:

```
/goal all tests in test/auth pass and the lint step is clean
```

The key is who decides whether it's met. Not the agent doing the work saying "I think that's good enough."

Official docs: After each turn, a small fast model checks whether the condition holds. If not, Claude starts another turn instead of returning control to you. completion is decided by a fresh model rather than the one doing the work.

After each turn, a small fast model checks whether the condition holds. If it doesn't, Claude runs another turn instead of handing control back to you. **Whether it's done is decided by a fresh model, not by the one doing the work.**

This is the maker-checker principle applied to the stop condition. The agent doing the work is the maker; that fresh model is the checker. It didn't take part in this round, carries no baggage of self-persuasion, and when you ask it "did the tests really pass," all it can do is go look at the test results. Underneath, it's a prompt-based Stop hook that intercepts each turn and refuses to let it through if the check fails.

Maker-checker wasn't invented by AI. When a bank moves a large sum, the person entering it and the person reviewing it have to be two different people. That rule has been around for decades, for exactly the same reason as here: the person who handled it will trust their own work, so the review only means

something if someone else does it. What /goal does is, at bottom, stuff that old rule into the agent's loop. Every "I'm done" has to clear a check that isn't its own.

	Who decides "done"	Problem / advantage
agent self-grades	the agent doing the work	carries self-persuasion, tends to praise itself
/goal	a fresh small model each turn	no baggage, can only look at objective conditions

One thing to keep straight. /goal is a Claude Code command name. Codex doesn't have this command; it reaches the same kind of capability through Automations plus agent configuration—same structure, different name. When you write them side by side, don't pin the command name on the wrong door.

And one that's even easier to confuse: don't mistake /loop for this. /loop reruns on a time interval—it's scheduling, a different thing from "run until the condition is met." The next chapter is all about it.

A loop's floor is its evaluator

Let me wrap this chapter. Building a loop that runs isn't hard. The hard part is giving it a reliable evaluator.

The generator's level decides what a loop can produce. **The evaluator's level decides what a loop won't produce.** Without the latter, the more cheerfully a loop runs, the more bad code it stacks up, because no one is calling stop.

Separate generation from judgment structurally, tune the evaluator into a skeptic, make it verify by acting rather than just reading, then hand the final say to a fresh model that didn't take part in the work. Those four steps are everything it takes to grow a loop's ability to say "no."

Next chapter we look at three loops that are actually running, and see how this whole thing works while everyone else is asleep.

§06 Loops That Run While You Sleep: Three Real Ones

Three Real Loops

Everything so far has been "how you should build one." This chapter only looks at "who actually built one and has it running." Three public cases, from one person's morning to a company shipping a thousand-plus PRs a week.

These three cases differ wildly in scale, but the skeleton is the same: a trigger presses start for you, a set of constraints keeps it from running off the rails, and a human checkpoint at the end. Once you've seen all three, you'll notice that "running while you sleep" was never about how strong the model is—it's about how solid that skeleton is.

1. Addy's morning: a triage loop that actually runs

Addy Osmani has described a loop of his own. Not a hypothetical—something that happens automatically every morning.

§ 03 already broke down the flow: at dawn an automation wakes itself up, reads CI and issues, opens a worktree, has sub-agents draft and review, opens a PR automatically when things pass, drops anything uncertain into an inbox for a human, and writes the state into a file for the next day. I'll only flag one detail § 03 didn't stress: **the automation invokes a skill, rather than gluing a huge block of instructions into the schedule.** Addy's line on this stings a bit.

Original: you fire \$skill-name instead of pasting a giant wall of instructions into a schedule that nobody will ever update.

You fire a \$skill-name, instead of pasting a giant wall of instructions into a schedule. Paste that in, and nobody's ever going to update it again.

This is what a loop one person can run looks like. One person, one machine, getting the grunt work done for you every morning.

2. Stripe's Minions: 1,300 PRs a week

To push a loop to enterprise scale, the case most worth studying is Stripe's Minions.

Put the number out front: **more than 1,300 PRs merged a week, not one line written by hand.** These details come from Stripe engineer Steve Kaliski on the podcast *How I AI*.

The trigger is light. You @ the Minion bot in Slack, or add an emoji reaction to a message, and then you're done thinking about it (fire-and-forget).

But what really makes it reliable isn't the light trigger—it's the stretch before the LLM wakes up. **Before the model starts thinking, a deterministic orchestrator assembles the context first:** it scans links in the message, pulls Jira, finds docs, and uses Sourcegraph plus MCP to search out the relevant code. By the time the agent takes the stage, all the materials it needs are already laid out on the table.

Why does this step matter? Because letting the LLM go find its own context is the least controllable part. It might search wrong, miss things, or make things up to fill in an answer. Stripe's move is to take "find the materials"—work whose rules you can hard-code—out of the LLM's hands and give it to deterministic code. The LLM only handles the part it's genuinely good at: writing code with all the materials in front of it. **Anything deterministic logic can solve never goes to a probabilistic model.** Where you draw that line directly decides whether the loop is reliable.

The most counterintuitive part is underneath. Minions isn't built on some stronger model. **It's a fork of the open-source tool Goose.** The core claim is one sentence: AI's reliability comes from the quality of the constraints, not the size of the model.

Its architecture is six layers, with deterministic gates and LLM creative steps interlocking in alternation. The agent finishes the code, a hard-coded pipeline runs the linter, and the agent can't skip past it; the agent fixes the lint; once fixed, a hard-coded step runs git commit. The sandbox is Devbox, running on EC2—"cattle not pets," used and discarded, not kept as a pet.

"Cattle not pets," that old ops line, fits perfectly here. Each agent's working environment is a head of cattle that can be swapped out anytime, not a cat you keep. If one breaks, you don't fix it—you spin up a new one. This design lets a thousand-plus agents run at once without stepping on each other, because no environment is "precious" and untouchable. The quality of the constraints shows up in these unglamorous engineering decisions, not in model parameters.

Step	Who does it
trigger	human (Slack @ or emoji)
assemble context	deterministic orchestrator (not the LLM)
write code	LLM agent
run linter / commit	hard-coded pipeline (agent can't skip)
review those 1,300 PRs	human

Note the last row. **The human didn't leave—the human changed desks.** Those 1,300 PRs are still reviewed by engineers; the time just moved from "writing" to "reviewing." Which lands right on the previous chapter's point: the more a loop can produce code you didn't write, the more critical the part that can say "no." Stripe hands half the "no" to hard-coded gates and leaves the other half to humans.

3. So what does "running while you sleep" actually rely on?

"Let the loop run while you sleep" is the sexiest line of this whole wave. But you have to say it precisely.

Local /loop, desktop scheduled tasks—**they all need your machine on.** Turn the machine off and the loop stops. If you really want it to run with the machine off, with you asleep, the right answer is something else: Cloud Routines, or GitHub Actions schedule triggers.

	Cloud Routines	Desktop scheduled task	/loop
where it runs	Anthropic cloud	your machine	your machine
machine on?	no	yes	yes
session open?	no	no	yes
min interval	1 hour	1 minute	1 minute
see local files?	no (fresh clone)	yes	yes

Reading this table is simple. **Want it frequent, want it to see local files? Use local /loop, at the cost of keeping the machine on. Want it to run with the machine off, not tied to local state? Go to the cloud, at the cost of a one-hour minimum interval and a clean clone every time.** No single scheduler does it all. Which one you pick depends on whether your loop is glued to the local machine or can leave it.

Two concrete scenarios. You want a loop to check a local dev server every minute—that can only be local /loop. The cloud can't see that process on your machine, and the interval can't go below a minute. Flip it: you want it to scan the repo's open issues at 3 a.m. every night and open PRs where warranted. That work shouldn't be tied to your laptop at all. Laptops get their lids closed, lose power, get carried out the door. For that, GitHub Actions schedule triggers or Cloud Routines are the right answer—they run on a machine that's always awake while you sleep.

"Running while you sleep" gets distorted so easily because a lot of people treat local /loop as the whole of it. Local /loop is "let it run a few extra rounds for me while I'm here." Cloud scheduling is "it runs even when I'm not." Two different things—don't mix them.

The scheduling itself isn't mysterious. /loop uses standard cron expressions, five fields, one-minute minimum granularity; if it annoys you, set CLAUDE_CODE_DISABLE_CRON=1 and it's all off. This layer has long been ops common sense. Loop engineering didn't reinvent it—it just hung an agent on top.

On the Codex side, the counterpart is the Automations tab. You can set daily, weekly, or use custom cron. It runs in a dedicated background worktree, and the results collect in a Triage inbox for you to look at when you're back. Same logic: a local automation also needs the machine on, running with it off relies on the cloud, and Codex's own cloud Jobs are still on the roadmap.

Put these three cases together and they form one line: an individual can build one (Addy's morning), a company can scale it (Stripe's thousand PRs), and there's a ready-made layer of scheduling to choose from (local or cloud). Loop engineering isn't a future concept. It's already on many people's machines and in many companies' CI, running every day.

A few widely circulated numbers are best treated with caution when you see them. Things like "around 90% of Claude Code is written by itself" or "Nubank migrated a million lines of ETL with a 12x speedup" are mostly secondhand summaries—take them as a rough reference, don't take them too seriously. The details of this chapter's three cases can all be traced to firsthand sources, and that holds up under scrutiny better than one impressive-sounding big number.

Next chapter we switch angles and look at the cost of all this. A loop running unattended is also a loop making mistakes unattended.

§07 The Costs: Verification Debt, Comprehension Rot, Token Blowout

The Costs

A loop doing your work for you sounds like all upside. But it's also quietly running up debt on your behalf. This chapter covers four tabs, and none of them clears itself.

A loop runs itself, and makes mistakes by itself

For several chapters I've been talking about how much a loop can do for you. Here I have to put out the other half. A loop that can run itself is, at the same time, a loop that can make mistakes by itself.

This line from Addy made me stop when I read it.

Original: A loop running unattended is also a loop making mistakes unattended.

A loop running unattended is also a loop making mistakes unattended. The more cheerfully it runs, the more quietly it errs.

The first tab is verification debt. You let the loop open PRs, change code, merge—every step saves you time. But the time you save doesn't come free. It turns into a pile of "not verified by anyone yet" output, sitting there waiting for you to pay it back.

The trouble with this debt is that it doesn't come for you right away. The loop changed twenty files tonight, the tests are all green, the PRs are all open—it looks gorgeous. The problem hides where the tests don't cover, in the gap between "runs" and "right." They lie there quietly, accumulating until some shipping morning when they all blow up at once. **You think you saved time; really you just pushed the day of reckoning back.**

Why does this debt build up? Because adding an agent is easy, and adding something that can stop an agent is hard. Addy puts this in strong terms.

Original: The hard part of a loop is not the loop. It is putting something inside it that can say no.

The hard part of a loop isn't the loop itself. It's putting something inside that can say "no." A loop without a real check is just an agent nodding at itself over and over.

How do you guard against it? Give the loop an independent evaluator—and that evaluator has to be a different one from the agent doing the work. A second agent, different instructions, ideally a different model, whose job is to pick faults. Letting the code-writing agent grade itself is letting it praise itself. This layer was already laid out earlier in generator and evaluator; verification debt is the bill when it isn't installed right.

You think you understand this repo—you don't anymore

The second tab is more hidden. Call it comprehension rot.

Every day the loop writes a pile of code you never wrote. The code runs, the tests pass, the PRs merge. On the surface, all normal. But something is slowly going off behind the scenes: your understanding of the project.

Original: The faster the loop ships code you did not write, the bigger the gap between what exists and what you actually get.

The faster the loop ships code you didn't write, the bigger the gap between "what actually exists" and "what you actually understand." Addy adds a line: whether your understanding rots depends on whether you let it rot.

Why does it happen? Because reading code is more boring than writing it, and the loop has taken over all the writing. You're no longer forced to read line by line, so naturally you stop. The codebase grows, but the map in your head is stuck three months ago. The day something goes wrong, you open the file and feel like you're looking at someone else's project.

The most treacherous thing about this gap is that it sounds no alarm in normal times. The project runs fine, and you feel like you understand it well enough. Until a bug burrows into a corner you've never read, the loop can't fix it, and you have to go in yourself. That's when you find out you've become a stranger to your own project. **Comprehension rot makes no sound. It only lets you discover, at the exact moment you most need to understand, that the understanding is already gone.**

How do you guard against it? Force yourself to read the loop's output regularly; don't just see "tests passed" and merge. Pick a few changes and explain to yourself what they did and why they did it that way. Can't explain it? That's your map needing an update.

When the loop runs smoothly, you stop bothering to have an opinion

The third tab is the psychological version of the first two. Call it cognitive surrender.

Original: When the loop runs itself its very tempting to stop having an opinion and just take whatever it gives back.

When the loop runs itself, it's very tempting to stop having an opinion and just take whatever it hands back. What this line hits is a state that's especially comfortable and especially dangerous.

Verification debt and comprehension rot are still technical problems. Cognitive surrender is an attitude problem. The first two are "you don't have time for it"; this one is "you don't want to bother anymore."

The more reliable the loop, the easier it is to outsource the whole job of judgment.

Why does it happen? Because having an opinion continuously is tiring. Thinking through whether every PR is right is far more tiring than just clicking "merge." The more steadily the loop runs, the more you tend to trust it; the more you trust it, the less you look; the less you look, the less able you are to judge. It's a self-accelerating slope.

How do you guard against it? Hold one line. The loop can execute for you; it can't decide for you. Whatever it hands back, you at least have to be capable of saying "this is wrong." You don't have to change it every time, but the capability has to be there. The day you can't say it, it's not that the loop got smart—it's that you quietly left, and no one's told you yet.

The bill may be a lot thicker than you think

The last tab is a cold-cash one. Call it token blowout.

The first three are implicit debt; this one is money hitting the bill directly. How much an autonomous loop consumes is hard to estimate accurately in advance.

Original: You absolutely have to be careful about token costs.

You absolutely have to be careful about token costs. Addy flags one key thing: usage swings wildly, depending on whether you're a token-rich person or a token-poor one. The same loop—the rich can afford to run it, the poor might wake up to a blown-up bill.

Why is it unpredictable? Because the loop hatches its own little helpers, retries on its own, and runs round after round on a timer. What you write is logic; what comes out is a count of runs, and the count times the unit price is the money. One bug spins it idle all night, and what you wake up to isn't fixed code—it's an unfamiliar bill.

How do you guard against it? Before you ship, set the loop a few hard caps: per-run budget, daily budget, max retries, and it stops when it hits the ceiling. These numbers aren't about saving money—they're about making sure an idle-spinning bug can't burn through an entire night's quota. **Don't wait for the bill to teach you this lesson. Nail the ceiling down with a number first.**

The four tabs together

These four tabs share one trait: none of them sounds an alarm while the loop is running.

Cost	Symptom	One line to guard against it
verification debt	output piles up unverified, errors accumulate quietly	install an evaluator that isn't the one doing the work
comprehension rot	code grows, the map in your head stalls	read the output regularly; can't explain it means update it
cognitive surrender	take whatever the loop gives, can't be bothered to have an opinion	execution can be outsourced, deciding can't
token blowout	usage swings wildly, bill unpredictable	nail down budget and retry caps before shipping

The most fascinating thing about loop engineering is that it lets one person do a team's work. The most dangerous thing is the same spot—a team reviews each other, reminds each other, argues with each other. One person plus a pile of loops easily turns into an echo chamber where no one argues.

These four tabs aren't trying to talk you out of loops. **They're a reminder that part of the time you saved has to go toward guarding these four gates for yourself.** Next chapter we talk about what guarding the gates actually relies on.

§08 Stay the Engineer, Not Just the One Who Presses Go

Stay the Engineer

The same loop, built by two different people, can end in opposite places. The difference isn't in the loop—it's in the hands that built it.

The same loop, opposite endings

The earlier chapters of this book taught a lot about how to build loops. This chapter I want to step back and talk about the person building them.

Addy writes one line in his piece that I think is the one most worth remembering in the whole thing.

Original: Two people can build the same loop and get opposite outcomes.

Two people build an identical loop and get completely opposite results.

This sounds a little counterintuitive. A loop is a system, a system is neutral—shouldn't the same code run out the same thing? But Addy isn't talking about the loop itself. He's talking about the person outside the loop.

One person uses a loop to move faster on things they've already mastered. They can read the code, they have a firm sense of direction, and the loop just takes the manual work off their hands. **What the loop scales for them is judgment they already had.**

Another person uses the same loop so they never have to understand again. Can't read it? Fine, the loop will write it. Can't judge it? Fine, the loop will merge it. They use the loop to route around the whole bothersome business of understanding.

Two people, the same tool—one accelerating understanding, one fleeing it. Look again six months later: one person has gotten stronger, the other has become the gatekeeper of a machine they don't even know what it's running.

This is worth thinking through one layer deeper. We always assume a tool's quality is decided by the tool—a strong tool yields strong results. But a loop isn't that kind of tool. **It's so strong that it amplifies, unchanged, whatever you bring into it.** Bring understanding, it amplifies understanding; bring laziness,

it amplifies laziness. The loop itself favors no side. It's just a faithful multiplication sign, and what it multiplies is you.

The plentiful and the scarce

The loop makes one thing extremely cheap: generation.

Code, plans, PRs, fixes—all can be mass-produced, almost free, almost no waiting. When something can be generated without limit, it stops being worth much. Today AI-written code is everywhere, and its scarcity is dropping to zero.

So what's still scarce?

Judgment. **Knowing which plan is right, which line of code should be stopped, which output runs fine but is wrong at the root—that's the thing the loop can't do for you.**

The loop can generate a hundred options, but it can't choose for you. Or more precisely: it can choose for you, but its basis for choosing is "looks reasonable," not "actually right." The gap between those two is exactly the reason an engineer exists.

So loop engineering doesn't devalue judgment—quite the opposite. It takes away all the work that doesn't need judgment, and what's left is all judgment. Your value gets compressed, distilled down to this one thing. Do it well and the loop amplifies you; do it badly and the loop amplifies your badness.

It's a pretty cruel amplifier, actually. In the past, a bad judgment meant at most you hand-wrote one wrong stretch of code—limited impact, still time to catch it. Now a bad judgment gets the loop to faithfully, in bulk, tirelessly execute that mistake a hundred times. It won't stop to ask you "is this really right." It'll just run faster. **The stronger the tool, the bigger the bet it places on your judgment.** You can no longer count on "the process is slow" to bail you out, because the process no longer has a slow gear.

The loop doesn't understand why you built it

There's one more layer, about who designs the loop.

The loop will faithfully execute the logic you give it. But it doesn't understand why you wanted to build it in the first place, doesn't understand what you actually want, doesn't understand which spots you'd really rather keep an eye on yourself. It only sees the code; it can't see the motive.

This means the loop's boundaries, where a manual checkpoint should sit, where it should be left to run on its own—none of these decisions can be read out of the loop. They only live in the head of the person who built it, and they have to be thought through in advance and written in one by one. Whatever you leave out, the loop won't fill in for you. It'll just run exactly as you wrote it, gaps you forgot to write and all.

The mindset you design it with is the shape it grows into. If you build it with a "free myself as fast as possible" mindset, you'll cut every check, because checks get in the way. If you build it with an "I still mean to be the engineer here" mindset, **you'll deliberately leave it a few gates where you can say no, even if that means it runs a little slower.**

Loops built with these two mindsets might be ninety percent identical in code; the difference is those one or two checkpoints. And it's exactly those one or two checkpoints that decide whether, six months out, you stand on top of the loop or get hollowed out by it. The first kind of person left a door, and can walk in anytime to see what's happening inside. The second kind welded the door shut, banking on never having to go in again. The day they have no choice but to go in, they find they no longer have the key.

The two endings of the same loop are buried right here, at the starting point.

Build the loop—but like someone who plans to stay

Addy's piece ends on one line. I'm putting it here unchanged, because I can't come up with a better closing.

Original: Build the loop. But build it like someone who intends to stay the engineer, not just the person who presses go.

Build the loop. But build it like someone who intends to stay the engineer, not just the person who presses go.

These two kinds of people look the same on the outside. Both sit outside the loop, both let agents do the work for them, both enjoy the rush of one person doing a team's work.

The difference is how they position themselves. One kind positions themselves as the engineer—the loop is a hand they've extended, and they always know what that hand is doing. The other kind positions themselves as a button: press it, wait for the result, and they can't quite say whether the result is good or bad.

Loop engineering gives everyone a chance to choose. You can use it to make yourself stronger, or use it to turn yourself into a redundant person. The tool is the same, and so is the button.

My own experience is that this choice isn't made once, the day you build the loop—it gets remade every day. Today you read one more PR, asked "is this really right" one more time, and you're still on the engineer's side. Today you took the easy way and merged with your eyes closed, and you slid one notch toward the button. **No one is born an engineer, and no one stays one once and for all. It's an identity you have to renew daily.**

What really decides the ending isn't how impressive a loop you built. It's whether, the moment you press go, you still count yourself as that engineer.

§09 Build Your First Loop Today

Build Your First Loop Today

After all that theory, this chapter does just one thing: walks you from a single command, layer by layer, to a loop that really runs on its own.

Don't aim for the finished product

A lot of people hear "loop engineering" and immediately picture Stripe's thousand-plus-PR-a-week pipeline, decide it's way out of reach, and give up.

That's a misunderstanding. Stripe's setup is the endpoint, not the starting point. **Your first loop should be so small it barely looks like a system—just a little thing that checks something for you on a timer.**

My suggestion is to start with a `/loop` scheduled task. It's a built-in Claude Code command, the lowest barrier there is, and the moment it runs you'll feel "oh, so that's what this is."

Step one: run a /loop

`/loop` is available after Claude Code v2.1.72, and it does one thing: on a time interval, rerun the same task. It has three forms—pick by need.

Three forms:

```
/loop 5m check the deploy : fixed, once every 5 minutes
```

```
/loop check the deploy : Claude sets its own pace (between 1 minute and 1 hour)
```

```
/loop : bare run, executing the built-in maintenance task or whatever you wrote in .claude/loop.md
```

The time units are s/m/h/d, and the minimum cron interval is 1 minute. Two details to keep in mind so you don't trip up.

First, it's session-scoped, and **recurring tasks expire after 7 days**—not 3, as some tutorials say. Second, it runs on your machine; turn the machine off and it stops. To really run while you sleep, with the machine off, you switch to Cloud Routines or GitHub Actions—more on that later.

Step two: have it read CI and issues, do triage first

Rerunning one line isn't a loop. You need it to read something and make a judgment. The most basic starting point is a triage task.

You give it a prompt to look at three things every morning: did yesterday's CI fail, what new issues were opened, what did recent commits change. Then it picks out what's worth handling and makes a list.

At this point you already have the two core parts of a loop: **a timer-triggered discovery mechanism, plus a triage brain that runs through the information for you.** Addy calls this kind of auto-discovery automation.

Addy says it's automation that makes a loop a real loop, rather than just the one time you ran it (his words are in § 03). That line names the threshold: scheduled plus auto-discovery is what gets you to loop entry level.

Step three: add a state file so it has memory

The triage runs—where do the results go? Don't leave them in the chat window; close it and they're gone. **Open a markdown file and write every finding, and how far each has been handled, into it.**

That file is the loop's memory. Addy puts it vividly: the agent forgets, the repo doesn't. Memory has to land on disk, not just live in the context. With it, work you didn't finish today, the loop can pick back up when it wakes tomorrow.

Don't want bare markdown? A Linear board or an issue list works too—same idea: state has to live outside any single conversation.

Step four: add an evaluator so it can say no

This step is the most critical in the whole loop, and the easiest to skip.

A loop nobody's watching makes mistakes nobody's watching too. You need to put something inside the loop that can say "no" to it, or it's just nodding at itself over and over.

Claude Code has a ready-made tool for exactly this called `/goal` (available after v2.1.139). It runs until some condition is met, and the one judging whether the condition holds is another model.

§ 05 broke down this mechanism: the one judging completion is another fresh model, not the one doing the work.

Why must it be a different model? Because the one doing the work is far too quick to praise itself. Finish the code, ask it to rate its own work, and it'll mostly say "looks good" with confidence. Only an evaluator with different instructions, even a different model, can catch the spots where it talked itself into glossing over. The usage is direct, for example:

```
/goal all tests in test/auth pass and the lint step is clean
```

Step five: add worktrees so it runs in parallel

With the first four steps built, you already have a single-threaded loop that runs on its own. The last step is letting it do several things at once.

The way is git worktree. Claude Code uses `--worktree` (or `-w`) to open an independent worktree for each background agent, so they don't step on each other.

Two agents writing the same file is as much of a headache as two engineers editing the same line (this is in § 04). Isolate them and they can run in parallel without fighting. At this point, your loop has all its vital organs.

Tool status at a glance (June 2026)

The commands above are all Claude Code's. Codex can do the same things—just different entry points and names. Don't pin the command names on the wrong house. `/loop` and `/goal` are Claude Code's, not Codex's.

Capability	Claude Code	Codex
scheduling	<code>/loop</code> (background timer worker)	Automations tab (daily/weekly + custom cron)
run until condition met	<code>/goal</code>	— (via automation rerun + judgment)
parallel isolation	<code>--worktree</code> / <code>-w</code>	dedicated background worktree, results to Triage inbox
sub-agents	Subagents / Agent Teams (<code>.claude/agents/</code>)	<code>.codex/agents/</code> TOML definitions
external connections	MCP + Plugins (one-click install)	MCP connector
explicit skill invocation	Skills (SKILL.md)	<code>\$skill-name</code>
runs with machine off	Cloud Routines (runs in the cloud)	cloud (planned Codex Jobs)

The MCP row is worth a word more: it's compatible across both sides. **A connector you write on one side can often be moved to the other and used directly.** As for Cursor's setup (Background Agents,

subagents, Cloud Agents), the capability path is similar, but my details on it are secondhand, so I won't go into it here—if you're going to use it, check its latest changelog yourself.

First-loop checklist

Before you start, run through these six. Whichever one you're missing, that's where your loop is at risk.

Element	Ask yourself
discovery source	what does it read on a timer? (CI / issues / commits / inbox)
state file	which disk file holds the cross-round memory?
evaluator	is there an independent check that can say "no"?
isolation	does each parallel agent get its own worktree?
token cap	did you set a spending ceiling? who stops it if it runs off?
human review point	which step pauses for you to look, rather than auto-ing all the way through?

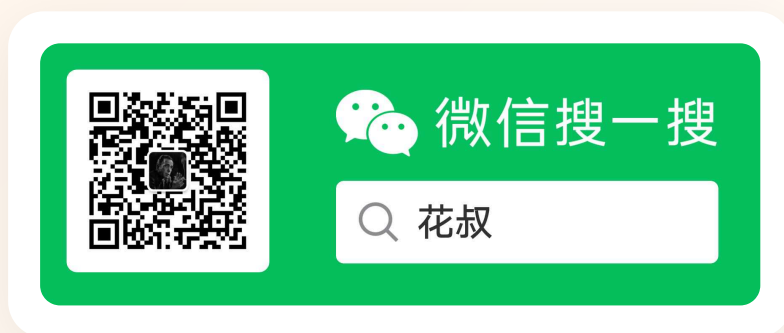
Of these six, the first two decide whether your loop can run; the last four decide whether it gets into trouble once it does. Beginners most often ship with just the first two built, and the result is a loop nobody's watching and nobody can stop, sitting there nodding at itself.

So my closing advice is simple: **your first loop—better small, but with that "no"-saying check and the human review point fully installed.** As for how to keep it from hollowing you out once it's built, § 08 already covered that—go back and flip through it.

This book ends here. Principles, parts, cases, costs—I've said everything there is to say. The rest isn't in the book; it's in your terminal. **Go build your first loop now.**

Loop Engineering

Stop Asking Me What It Is



HuaShu • AI Native Coder

I can't write a single line of code, yet I built an App Store #1 paid app
and several technical books — all with AI.

I only decide what to build. I believe AI-generated content isn't
scarce; judgment is.

[Bilibili](#) • [WeChat: 花叔](#) • [X: @AlchainHust](#) • [YouTube](#) • [GitHub](#) • [huasheng.ai](#)

Created by HuaShu • June 2026

All Orange Books free to download: huasheng.ai/orange-books